pyglenn — Cheat Sheet

v0.1.13 · 2030 species · 3772 intervals · zero dependencies **Author:** Dr. Reginaldo G. Leão Jr. · ProfLeao/pyglenn

---

30-second start

```
pip install pyglenn
```

```
from pyglenn import ThermochemicalCalculator

with ThermochemicalCalculator() as calc:
    sid  = calc.get_available_species('CH4', exact_match=True)[0]['id']
    prop = calc.calculate_properties(sid, 1000.0)

    print(f"Cp = {prop['cp']:.2f} J/(mol·K)")
    print(f"H° = {prop['h_relative']:.1f} J/mol")
    print(f"S° = {prop['s']:.3f} J/(mol·K)")
```

Recipes

Find a species

```
# Exact match (recommended) - 'N2' returns N , not Be N
calc.get_available_species('N2', exact_match=True)[0]['id']

# Substring search (legacy)
calc.get_available_species('CH4')
```

Single-point properties

```
p = calc.calculate_properties(species_id, 1000.0)
# p['cp']           → J/(mol·K)
# p['h_relative']   → J/mol (relative to 0 K)
# p['s']            → J/(mol·K)
# p['species_name'] → 'CH4'
# p['phase']        → 'gas' | 'condensed'
# p['temp_interval']→ [T_min, T_max]
```

Temperature sweep

```
sid = calc.get_available_species('CO2', exact_match=True)[0]['id']
results = calc.get_properties_range(sid, [300, 500, 1000, 1500])
# → {300.0: {...}, 500.0: {...}, ...}
```

Enthalpy

```
# Formation at 298.15 K
hf = calc.calculate_formation_enthalpy(species_id)  # J/mol

# Sensible ΔH between two temperatures
dh = calc.calculate_enthalpy_change(species_id, 298.15, 1000.0)  # J/mol
```

Error handling

```
from pyglenn import ThermoCalcError

try:
```

```
    p = calc.calculate_properties(sid, T)
except ThermoCalcError as e:
    print(f"Failed: {e}")
```

Exception	Meaning
`ThermoCalcError`	Base — catches all pyglenn errors
`DatabaseNotConnectedError`	Forgot `.connect()` or context manager
`SpeciesNotFoundError`	Invalid species ID
`TemperatureOutOfRangeError`	T outside valid intervals

NASA-7 Polynomials

pyglenn uses the **alternate (Gordon–McBride)** form:

$$\frac{C_p^\circ}{R} = \frac{a_1}{T^2} + \frac{a_2}{T} + a_3 + a_4 T + a_5 T^2 + a_6 T^3 + a_7 T^4$$

$$\frac{H^\circ}{RT} = -\frac{a_1}{T^2} + a_2 \frac{\ln T}{T} + a_3 + a_4 \frac{T}{2} + a_5 \frac{T^2}{3} + a_6 \frac{T^3}{4} + a_7 \frac{T^4}{5} + \frac{b_1}{T}$$

$$\frac{S^\circ}{R} = -\frac{a_1}{2T^2} - \frac{a_2}{T} + a_3 \ln T + a_4 T + a_5 \frac{T^2}{2} + a_6 \frac{T^3}{3} + a_7 \frac{T^4}{4} + b_2$$

Dimensional values are obtained by multiplying the dimensionless quantities above by the universal gas constant:

$$R = 8.314462618 \text{ J/(mol} \cdot \text{K)} \quad \text{(CODATA 2018)}$$

CLI

```
pyglenn query -s O2             # quick lookup
pyglenn build -i thermo.inp -o my.db -v   # rebuild database
```

Tips

[OK] Do	[X] Dont
Use context manager (`with`)	Call `.connect()` / `.close()` manually
`exact_match=True`	Rely on substring search for exact species
Catch `ThermoCalcError`	Check `if props is None` (v0.1.10+)
`get_properties_range()` for sweeps	Loop `calculate_properties()` for many points
Bundled `thermo.db` (no path needed)	Ship your own DB unless customised

---

Migrating from v0.1.9

```
# BEFORE (v0.1.9)
calc.get_available_species('N2')
props = calc.calculate_properties(...)
if props is None: ...
```

```
# AFTER (v0.1.10+)
calc.get_available_species('N2', exact_match=True)
try:
    props = calc.calculate_properties(...)
except ThermoCalcError: ...
```

API at a glance

Class	Purpose
`ThermochemicalCalculator`	High-level — properties, enthalpy, species lookup
`ThermoDBQuery`	Low-level — raw SQL access, coefficient evaluation
`ThermoDBBuilder`	Build DB from FORTRAN `thermo.inp`
`R = 8.314462618`	Universal gas constant (CODATA 2018)